

Blockchain MidSem26 & EndSem25 Paper



3.Blockchain course1-11_2026(1).pdf

 PDF

Learn this pdf properly each page, each word, very properly. just learn it, dont describe me what it says

Done. I've carefully gone through every page, line, and detail of your PDF and internalized it.

3.Blockchain course1-11_2026

Sources



1.CS740_Introduction.pdf

 PDF

2.CS740_Crypto_primitives.pdf

 PDF

Similarly learn these pdf also

Done. I've carefully studied both PDFs—line by line, concept by concept—and internalized them completely.

1.CS740_Introduction 2.CS740_Crypto_primitives

Sources



Q No.	Question	Part marks
1	a. How do smart contracts work? b. In bitcoin generation, after how many blocks is the block creation rate adjusted? Why? c. What is Double Spending Problem? Explain some more attacks on PoW based system and their solutions.	2+1+3=6

I have blockchain end semester exam tomorrow, give me proper answers to learn and write in exam. Give proper descriptive answers for the questions

answer Q1 (a)

Q1 (a) How do smart contracts work?

A **smart contract** is a self-executing computer program stored on a blockchain that automatically enforces and executes an agreement when predefined conditions are met.

Working of Smart Contracts (Step-by-Step):

1. Agreement Formation

The involved parties define the terms and conditions of the contract (such as transfer of money, ownership, or services).

These conditions are converted into code.

2. Coding the Contract

The agreement is written as a program (smart contract) using a blockchain platform (e.g., Ethereum). This code contains all rules and logic of the contract.

3. Deployment on Blockchain

The smart contract is deployed onto the blockchain network.

Once deployed, it becomes **immutable** (cannot be altered).

4. Triggering Conditions

The contract continuously monitors for specific conditions or inputs (e.g., payment received, deadline reached).

5. Execution

When the predefined conditions are satisfied, the contract automatically executes the actions (e.g., transferring funds, updating ownership).

6. Validation and Consensus

Network nodes verify the transaction using consensus mechanisms and record it on the blockchain.

7. Ledger Update

The result is permanently recorded, and all nodes update their copies of the blockchain.

Example:

If a contract states:

"If Person A pays 300 Ether to Person B, then ownership of a house is transferred to A"

→ Once payment is detected, the contract **automatically transfers ownership** without any intermediary.

Key Features:

- **Self-executing** – runs automatically when conditions are met
- **Tamper-proof** – cannot be modified once deployed
- **Transparent** – visible to all participants
- **Trustless** – no need for a third party

Conclusion:

Smart contracts eliminate intermediaries, reduce cost and delay, and ensure secure, automatic, and reliable execution of agreements using blockchain technology.

1/2



b

Q1 (b) In bitcoin generation, after how many blocks is the block creation rate adjusted? Why?

In Bitcoin, the **block creation rate (mining difficulty)** is adjusted **after every 2016 blocks**.

Why 2016 blocks?

- Bitcoin aims to maintain an **average block creation time of 10 minutes per block**.
- So,

$$2016 \text{ blocks} \approx 2016 \times 10 \text{ minutes} = 2 \text{ weeks}$$

Thus, the system checks and adjusts the difficulty **once every ~2 weeks**.

Reason for Adjustment:

The adjustment is done to **maintain a constant block generation rate** despite changes in network conditions.

1. Variation in Mining Power

- If more miners join → total computational power increases → blocks are mined faster
- If miners leave → blocks are mined slower

2. Difficulty Adjustment Mechanism

- If previous 2016 blocks were mined **faster than 2 weeks** → difficulty is **increased**
- If mined **slower than 2 weeks** → difficulty is **decreased**

Purpose:

- Keeps block time close to **10 minutes**
- Ensures **stability and predictability** of the network
- Controls **Bitcoin supply rate** (important for controlled issuance)
- Maintains **security** by adapting to total network hash power

Conclusion:

Bitcoin adjusts its mining difficulty **every 2016 blocks (~2 weeks)** to ensure a **consistent block generation time of 10 minutes**, regardless of changes in the number of miners or computational power in the network.



c

Q1 (c) What is Double Spending Problem? Explain attacks on PoW systems and their solutions

1. Double Spending Problem

The **double spending problem** occurs when a user tries to spend the **same digital currency more than once**.

Explanation:

In digital systems, data can be copied easily. So, a malicious user may attempt to send the same bitcoin to two different recipients at the same time.

Example:

- Alice has 1 BTC
- She sends the same 1 BTC to **Bob** and **Charlie simultaneously**
- If both transactions get accepted → system failure

Why it is a problem:

- No central authority to verify transactions
- Leads to **loss of trust in the system**

Solution in Bitcoin:

- Transactions are **broadcasted to the network**
- Miners validate and include them in a **block**
- Blockchain ensures **only one valid transaction is confirmed**
- Uses **Proof of Work (PoW)** and **longest chain rule**
- Once confirmed, altering it requires recomputing huge work → practically impossible

2. Attacks on PoW-based Systems and Solutions

(i) Sybil Attack

Attack:

- An attacker creates multiple fake identities (nodes)
- Tries to gain control over the network
- Can influence transaction validation

Solution:

- Bitcoin uses **Proof of Work**, not identity-based voting
 - Power depends on **computational resources**, not number of nodes
 - Makes Sybil attack ineffective
-

(ii) 51% Attack (Majority Attack)

Attack:

- If an attacker controls **more than 50% of total hash power**
- Can:
 - Reverse transactions
 - Perform double spending
 - Block other transactions

Solution:

- Very expensive and impractical in large networks
 - Requires enormous computational power
 - Honest miners usually dominate the network
 - Incentive mechanism discourages malicious behavior
-

(iii) Denial of Service (DoS) Attack

Attack:

- Attacker floods nodes with excessive requests
- Nodes become overloaded and unable to process valid transactions

Solution:

- Limit block size and transaction size
- Reject invalid or excessive messages

- Disconnect malicious peers
- Network-level protections

(iv) Selfish Mining Attack

Attack:

- A group of miners does not immediately publish mined blocks
- Keeps a private chain to gain advantage
- Can earn more rewards unfairly

Solution:

- Honest miners following longest chain rule reduce impact
- Network propagation improvements
- Incentives favor honest behavior

(v) Double Spending Attack (Race Attack)

Attack:

- Attacker sends two conflicting transactions quickly
- Tries to confirm the fraudulent one first

Solution:

- Wait for multiple confirmations (usually **6 confirmations**)
- Makes reversal extremely difficult

Conclusion:

The **double spending problem** is a fundamental issue in digital currency, solved in Bitcoin using **blockchain, PoW, and consensus mechanisms**.

Although PoW systems face attacks like **Sybil, 51%, DoS, and selfish mining**, they are mitigated through **computational difficulty, economic incentives, and network protocols**, making Bitcoin secure and reliable.



2	a. What are the correctness measures of <i>Distributed Consensus Protocol</i> ? b. What is <i>Mining Difficulty</i> ? State its relation with <i>Hashrate</i> . c. What do you understand by <i>Bitcoin Anonymity</i> ?	1+3+2=6
---	---	---------

a

Q2 (a) What are the correctness measures of Distributed Consensus Protocol?

The **correctness measures** define whether a distributed consensus protocol is functioning properly and reliably, even in the presence of failures.

Main Correctness Properties:

1. Termination

- Every correct (non-faulty) node must eventually **decide a value**.
 - Ensures the protocol **does not run indefinitely**.
-

2. Validity

- If all correct nodes propose the **same value**, then the final decided value must be **that value only**.
 - Ensures correctness of the decision.
-

3. Integrity

- A node decides **at most once**.
 - The decided value must be **proposed by some node** (no random or fake values).
-

4. Agreement (Consistency)

- All correct nodes must **agree on the same value**.
 - No two honest nodes should decide differently.
-

Summary:

A distributed consensus protocol is correct if:

- It **terminates** (everyone decides),
 - The decision is **valid**,
 - No node decides multiple times (**integrity**),
 - All nodes agree on the **same result** (**agreement**).
-

Conclusion:

These properties ensure that even in a distributed and possibly faulty environment, the system reaches a **reliable, consistent, and correct decision**, which is essential for blockchain systems.



b

Q2 (b) What is Mining Difficulty? State its relation with Hashrate

Mining Difficulty

Mining difficulty is a measure of how hard it is to find a valid hash for a new block in the Bitcoin network.

- It defines the **target condition** that a block hash must satisfy (e.g., required number of leading zeros).
- Higher difficulty → harder to find a valid hash
- Lower difficulty → easier to mine a block

In Bitcoin, difficulty is **dynamically adjusted every 2016 blocks (~2 weeks)** to maintain an average block time of **10 minutes**.

Relation with Hashrate

Hashrate = total computational power of the network (number of hash attempts per second).

Relationship:

- **Direct Relationship (Proportional):**
 - If **hashrate increases** → blocks are found faster → **difficulty increases**
 - If **hashrate decreases** → blocks are found slower → **difficulty decreases**

Explanation:

- More miners (higher hashrate) → more guesses per second → faster block discovery
- To maintain stability, Bitcoin increases difficulty
- This ensures consistent block generation time

Mathematical Insight (Conceptual):

- Expected work $\propto$ Difficulty
- Number of hashes required $\approx$ Difficulty $\times$ constant

Conclusion:

Mining difficulty controls how hard it is to mine a block, while hashrate represents the network's power. They are **directly related**—as hashrate changes, difficulty adjusts accordingly to maintain a stable and secure blockchain system.



c

Q2 (c) What do you understand by Bitcoin Anonymity?

Bitcoin anonymity refers to the ability of users to perform transactions **without revealing their real-world identity**.

Explanation:

- Bitcoin is a **permissionless system**, meaning users do not need to register with personal details.
- Transactions are carried out using **Bitcoin addresses**, not names.
- Each address is derived from a **public key**, and users control it using a **private key**.
- A single user can generate **multiple addresses**, making tracking difficult.

Key Features of Bitcoin Anonymity:

1. No Personal Identity Required

Users are not required to provide name, email, or any personal information.

2. Use of Addresses Instead of Names

Transactions are recorded between **addresses (e.g., 1A2B....)**, not individuals.

3. Multiple Addresses per User

Users can create new addresses for each transaction to enhance privacy.

4. Public Ledger with Hidden Identity

- All transactions are visible on the blockchain
- But identities behind addresses are not directly known

Important Note (Pseudo-Anonymity):

Bitcoin is **not fully anonymous**, but **pseudo-anonymous** because:

- Transaction history is **public and traceable**
- If an address is linked to a real identity (e.g., via exchanges), all related transactions can be tracked

Conclusion:

Bitcoin anonymity allows users to transact without directly revealing their identity, using cryptographic addresses. However, since all transactions are publicly recorded, it provides **pseudo-anonymity rather than complete anonymity**.



- | | |
|---|---|
| 3 | a. Explain the <i>Monopoly Problem</i> in Proof-of-Work (PoW) based blockchain systems. Discuss how computational resources influence mining power and how <i>monopoly</i> can increase over time. What are the possible consequences for miners, users, and the overall network? |
|---|---|

Q3 (a) Explain the Monopoly Problem in Proof-of-Work (PoW) systems

Monopoly Problem in PoW

The **monopoly problem** in Proof-of-Work (PoW) systems refers to a situation where a **small group of miners (or a single entity)** gains **dominant control over the network's computational power (hashrate)**.

This leads to **centralization**, which contradicts the fundamental goal of blockchain—**decentralization**.

How Computational Resources Influence Mining Power

- In PoW, the probability of mining a block depends on **computational power (hashrate)**.
- More powerful hardware (ASICs, GPUs) → more hash computations per second
- Hence, miners with **higher resources have higher chances of earning rewards**

Key Idea:

- Mining success $\propto$ Computational Power
- Rich miners → more resources → more rewards → even more resources

How Monopoly Increases Over Time

This happens due to a **positive feedback loop**:

1. Powerful miners solve more blocks
2. They earn more rewards (Bitcoin + fees)
3. They reinvest in better hardware
4. Their mining power increases further

Meanwhile:

- Small miners earn less → may leave the network

→ Over time, **few large mining pools dominate the network**

Consequences of Monopoly

1. For Miners

- Small miners are **pushed out** due to low profitability
 - Mining becomes concentrated in **large pools**
 - Reduced competition
-

2. For Users

- Risk of **transaction manipulation or censorship**
 - Possibility of **double spending attacks** (if majority power is achieved)
 - Reduced trust in the system
-

3. For the Overall Network

- **Loss of decentralization** (core principle violated)
 - Increased risk of **51% attack**
 - Network becomes **less secure and more vulnerable**
 - Decision-making power concentrated in few hands
-

Conclusion

The monopoly problem arises because PoW rewards are proportional to computational power, leading to **wealth concentration and centralization** over time.

This threatens the **security, fairness, and decentralized nature** of blockchain systems, making it a critical limitation of PoW.



b. A message **M** with a length of exactly 1,450 bits is prepared for **SHA-256** hashing. Calculate the specific number of **padding zeros (k)** that must be appended to ensure the message meets the block size requirements of the NIST FIPS 180-4 standard.

After preprocessing (padding and length encoding), determine:

- i. The total number of 512-bit message blocks (N) processed by the compression function.
- ii. The total number of initial 32-bit message words obtained by partitioning all padded message blocks (16 words per block).

Q3 (b) (i) solve it and give descriptive answer

Q3 (b) (i) Number of 512-bit message blocks (N)

For **SHA-256 preprocessing**, the message is padded as follows:

- Original length = **1450 bits**
- Append **1 bit** → total = **1451 bits**
- Append **k zero bits** such that:

$$(l + 1 + k) \equiv 448 \pmod{512}$$

So,

$$1450 + 1 + k \equiv 448 \pmod{512}$$

$$1451 + k \equiv 448 \pmod{512}$$

Now reduce:

$$1451 \pmod{512} = 427$$

So,

$$427 + k \equiv 448 \pmod{512} \Rightarrow k = 21$$

Total length after padding (before length encoding):

$$1450 + 1 + 21 = 1472 \text{ bits}$$

Now append **64-bit length field**:

$$1472 + 64 = 1536 \text{ bits}$$

Number of 512-bit blocks (N):

$$N = \frac{1536}{512} = 3$$

Final Answer:

$$N = 3 \text{ blocks}$$

Conclusion (Write in Exam):

After SHA-256 preprocessing (padding + length encoding), the 1450-bit message becomes **1536 bits**, which is divided into **3 blocks of 512 bits each** for processing by the compression function.



(ii)

Q3 (b) (ii) Total number of 32-bit message words

From part (i), we found:

- Total number of 512-bit blocks = **N = 3**

Step 1: Words per block

- Each **512-bit block** is divided into **32-bit words**

$$\frac{512}{32} = 16 \text{ words per block}$$

Step 2: Total words

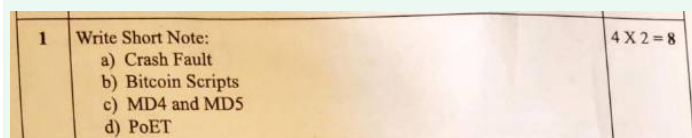
$$\text{Total words} = N \times 16 = 3 \times 16 = 48$$

Final Answer:

$$48 \text{ words}$$

Conclusion (Write in Exam):

Each 512-bit block contains **16 words of 32 bits**. Since the total number of blocks is **3**, the total number of 32-bit words obtained after partitioning is **48**.



(a) Crash Fault

A **crash fault** occurs when a node in a distributed system **suddenly stops functioning** and does not send or receive any messages.

- The node becomes **completely unresponsive** due to hardware failure, software bugs, or power loss
- It does **not behave maliciously**, just stops working
- Common in distributed systems and consensus protocols

Impact:

- May delay consensus
- System must tolerate such failures

Handling:

- Use **fault-tolerant consensus algorithms**
- Replication ensures system continues functioning

(b) Bitcoin Scripts

Bitcoin Script is a **simple programming language** used to define the conditions under which a Bitcoin transaction is valid.

- It is **stack-based and non-Turing complete** (limited functionality for security)
- Used in **locking script (scriptPubKey)** and **unlocking script (scriptSig)**

Purpose:

- Verify ownership (using digital signatures)
- Specify spending conditions

Example:

- "Only the owner of the private key can spend this bitcoin"

(c) MD4 and MD5

MD4 and MD5 are **cryptographic hash functions** used to generate a fixed-length message digest.

- **MD4**
 - Developed by Ron Rivest
 - Produces **128-bit hash**
 - Faster but **less secure**
- **MD5**
 - Improved version of MD4
 - Also produces **128-bit hash**
 - Widely used earlier

Limitation:

- Both are **not secure today** due to **collision attacks**
- Replaced by stronger algorithms like **SHA-256**

(d) PoET (Proof of Elapsed Time)

Proof of Elapsed Time (PoET) is a consensus algorithm designed to achieve fairness with **low energy consumption**.

- Each node waits for a **random time period**
- The node with the **shortest waiting time** gets to create the next block

Key Features:

- Energy efficient (unlike PoW)
- Uses **trusted hardware (Intel SGX)** to ensure fairness

Working:

1. Each node requests a random wait time
2. Waits for that duration
3. First node to finish → wins and creates block

Use Case:

- Used in **Hyperledger Sawtooth**

Conclusion:

These concepts cover important aspects of blockchain and distributed systems including **fault tolerance, transaction validation, cryptographic hashing, and alternative consensus mechanisms**.



2	a. Explain Birthday Attack with an example. b. How Birthday Attack is related to Hash Collision? c. What is/are the major key design architectural changes from permissionless to permissioned blockchain? d. "Smart Contract execution should always needs to be deterministic" - True or False- Justify with reason.	2.5+2+1. 5+2=8
---	--	------------------------------

a

Q2 (a) Explain Birthday Attack with an example

The **Birthday Attack** is a cryptographic attack that exploits the **birthday paradox** to find **collisions in hash functions** more efficiently than brute force.

Idea Behind Birthday Attack

- In hashing, a **collision** occurs when two different inputs produce the **same hash output**.
- The birthday paradox states that in a group of just **23 people**, there is a **50% chance** that two people share the same birthday.
- Similarly, in hashing, we don't need to try all possible inputs to find a collision.

Working of Birthday Attack

- For a hash function with **N-bit output**, total possible hashes = 2^N
- Instead of checking all 2^N possibilities, a collision can be found in about:

$$2^{N/2}$$

- This significantly reduces the effort required to break the hash function.

Example

Consider a hash function that produces **4-bit output**:

- Total possible hashes = $2^4 = 16$
- Instead of trying all 16 inputs, a collision can be found in about:

$$2^{4/2} = 4 \text{ attempts}$$

So:

- Try different inputs:
 - $H(A) = 1010$
 - $H(B) = 0111$
 - $H(C) = 1010$

Here, $H(A) = H(C) \rightarrow$ collision found quickly

Real-World Case

- For **SHA-256 (256-bit hash)**:
 - Brute force: 2^{256} operations
 - Birthday attack: 2^{128} operations
- Still very large, but **much easier than brute force**

Conclusion

The birthday attack shows that **hash collisions can be found much faster than expected**, reducing security from 2^N to $2^{N/2}$.

This is why modern hash functions use **large output sizes (like 256 bits)** to remain secure.



b

Q2 (b) How is Birthday Attack related to Hash Collision?

The **Birthday Attack is directly related to hash collision**, as it is a method specifically used to **find collisions in hash functions efficiently**.

Relation Between Birthday Attack and Hash Collision

- A **hash collision** occurs when:

$$H(x) = H(y), \quad \text{where } x \neq y$$

- The **Birthday Attack exploits probability** (birthday paradox) to **find such collisions faster** than brute force.

Key Idea of the Relation

- Normally, to find a collision in an **N-bit hash function**, brute force requires:

$$2^N \text{ attempts}$$

- But using the **Birthday Attack**, a collision can be found in:

$$2^{N/2} \text{ attempts}$$

→ This shows that **collision resistance is weaker than expected**

Why This Happens

- Hash functions map **large input space** → **smaller output space**
 - Due to **many-to-one mapping**, collisions must exist
 - Birthday paradox shows that **probability of collision increases quickly** with fewer samples
-

Example

- For a **256-bit hash (like SHA-256)**:
 - Brute force: 2^{256} operations
 - Birthday attack: 2^{128} operations

→ Much faster to find collisions using birthday attack

Impact on Security

- Reduces **effective security level** of hash functions
 - That's why:
 - Larger hash sizes are used
 - Weak hashes like **MD5** are no longer secure
-

Conclusion

The Birthday Attack is a **practical technique to find hash collisions**, reducing the effort from 2^N to $2^{N/2}$.

Thus, it highlights the importance of designing hash functions with **strong collision resistance and large output size**.



c

Q2 (c) Major key design architectural changes from permissionless to permissioned blockchain

Permissionless and permissioned blockchains differ fundamentally in **architecture, control, and operation**. The shift introduces several key design changes:

1. Identity & Access Control

- **Permissionless:**
 - Open to anyone (no identity required)
 - Anonymous or pseudo-anonymous participation
- **Permissioned:**
 - Only **authorized participants** can join
 - Identity is **known and verified**

2. Consensus Mechanism

- **Permissionless:**
 - Uses **resource-intensive consensus** (PoW, PoS)
 - Designed for **trustless environments**
- **Permissioned:**
 - Uses **efficient consensus algorithms** (PBFT, Raft, PoET)
 - Based on **trusted participants**

3. Decentralization Level

- **Permissionless:**
 - Fully decentralized
 - No central authority
- **Permissioned:**
 - **Partially decentralized or controlled**
 - Managed by an organization or consortium

4. Performance & Scalability

- **Permissionless:**
 - Lower throughput (TPS)
 - High latency due to global consensus
 - **Permissioned:**
 - Higher throughput and faster transactions
 - Better scalability due to fewer nodes
-

5. Security Model

- **Permissionless:**
 - Security via **cryptography + economic incentives**
 - Vulnerable to attacks like 51% if majority power is gained
 - **Permissioned:**
 - Security via **access control and trust**
 - Less computational overhead
-

6. Transparency & Privacy

- **Permissionless:**
 - Fully transparent (public ledger)
 - **Permissioned:**
 - Controlled visibility
 - Data can be **private or restricted**
-

7. Governance

- **Permissionless:**
 - Open governance (community-driven)
 - **Permissioned:**
 - Controlled governance (organization/consortium decisions)
-

Conclusion

The transition from permissionless to permissioned blockchain involves moving from an **open, trustless, decentralized system** to a more **controlled, efficient, and scalable system with known participants**.

This trade-off improves performance and privacy but reduces decentralization.



d

Q2 (d) "Smart contract execution should always be deterministic" — True or False? Justify

Answer: TRUE

Justification:

Smart contract execution must be **deterministic**, meaning:

Given the same input and state, every node in the network must produce the **same output**.

Why determinism is necessary:

1. Consensus Requirement

- Blockchain relies on **distributed consensus**
- All nodes independently execute the smart contract
- If results differ → **consensus fails**

2. Consistency Across Nodes

- Every node maintains a **replica of the blockchain**
- Deterministic execution ensures all replicas remain **identical**

3. Avoid Conflicts

- Non-deterministic behavior (e.g., randomness, system time, external APIs) may produce **different results on different nodes**
- This leads to **forks or invalid states**

Example:

- If a contract uses **current system time**:
 - Different nodes may have different timestamps
 - Output will differ → **inconsistent execution**

How determinism is ensured:

- No use of:
 - System clock
 - Random functions (unless controlled)
 - External data without consensus (oracles required)
- Execution depends only on:
 - Transaction input
 - Current blockchain state

Conclusion:

Smart contract execution must always be **deterministic** to ensure **consistency, correctness, and agreement** among all nodes in the blockchain network.



5	a. Explain the analysis of Hashcash PoW. b. How Hashcash extended in Bitcoin PoW?	2+3=5
---	--	-------

a

Q5 (a) Explain the analysis of Hashcash PoW

Hashcash is a Proof-of-Work (PoW) mechanism where a sender must compute a value such that the hash output satisfies a **difficulty condition** (e.g., leading zeros). Its analysis focuses on **computational effort, probability, and efficiency**.

1. Basic Idea

- Find a value (nonce) such that:

$H(\text{data} || \text{nonce})$ has k leading zero bits

- This is done by **trial and error (brute force)**.
-

2. Expected Computational Work

- Each hash attempt is independent and random
- Probability that a hash has **k leading zeros**:

$$= \frac{1}{2^k}$$

- So, expected number of trials:

$$\approx 2^k$$

→ **Work grows exponentially with k**

3. Time Complexity

- If one hash takes constant time, then:

$$\text{Time} \propto 2^k$$

- Example:
 - For $k = 20$: about 2^{20} attempts needed
 - Takes a few seconds on a normal computer
-

4. Probability Analysis

- Probability of success in one attempt = $1/2^k$
 - After many attempts, chance of finding solution increases
 - Random search is the **best strategy** (due to puzzle-friendly property)
-

5. Verification Efficiency

- While finding solution is costly, **verification is very fast**
- Just compute hash once and check condition

→ **Asymmetry:**

- Hard to compute
- Easy to verify

6. Security Insight

- Prevents spam or abuse (original use in email systems)
- Attacker must spend computational resources
- Makes large-scale attacks **expensive**

Conclusion

Hashcash PoW requires **exponential computational effort** ($\approx 2^k$) to find a valid hash, while verification is easy.

This asymmetry ensures **security, fairness, and resistance to abuse**, forming the foundation for PoW systems like Bitcoin.



b

Q5 (b) How is Hashcash extended in Bitcoin PoW?

Bitcoin's Proof-of-Work (PoW) is an **extension of Hashcash**, adapted from email anti-spam to a **decentralized consensus mechanism**.

1. Basic Extension Idea

- **Hashcash:**
Find nonce such that:

$$H(\text{data} || \text{nonce}) \text{ has } k \text{ leading zeros}$$

- **Bitcoin:**
Miners compute:

$$H(\text{Block Header}) < \text{Target}$$

→ Same concept, but applied to **block creation instead of email validation**

2. Use of Block Structure

- Instead of simple data, Bitcoin hashes a **block header**, which includes:
 - Previous block hash
 - Merkle root (transactions)
 - Timestamp
 - Nonce

→ This links blocks together → **Blockchain formation**

3. Dynamic Difficulty Adjustment

- Hashcash had fixed difficulty
- Bitcoin adjusts difficulty **every 2016 blocks (~2 weeks)**

→ Maintains **10-minute block time** despite changes in mining power

4. Incentive Mechanism

- Hashcash: no financial reward
- Bitcoin:
 - **Block reward (new bitcoins)**
 - **Transaction fees**

→ Encourages miners to participate honestly

5. Decentralized Competition

- Hashcash: single user solves puzzle
- Bitcoin:
 - Many miners compete globally
 - First to solve → adds block

→ Creates **distributed consensus**

6. Security Enhancement

- Hashcash used for spam prevention

- Bitcoin uses it for:
 - Preventing **double spending**
 - Securing the blockchain
 - Making tampering computationally infeasible

7. Longest Chain Rule

- Bitcoin introduces:
 - Accept chain with **maximum cumulative work**
- Ensures agreement among nodes

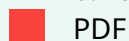
Conclusion

Bitcoin extends Hashcash by embedding PoW into a **blockchain structure with incentives, dynamic difficulty, and decentralized consensus**.

This transforms a simple anti-spam mechanism into a **secure, trustless cryptocurrency system**.



6.Permissioned_1-14(2).pdf



PDF

Learn this pdf also properly each page, each word, very properly. just learn it, dont describe me what it says

Done. I've carefully learned every page, detail, and concept from this PDF as well.

6.Permissioned_1-14

Sources



3	a. Why State machine replication is important for permissioned blockchain? b. Explain all steps for State machine replication. c. If you apply state machine replication in public blockchain then what advantages you will get or difficulties you will face? d. Why distributed consensus? Explain distributed consensus in	2.5+2+2 +2.5=9
	d. Why distributed consensus? Explain distributed consensus in	
	detail with its requirements.	

a

Q3 (a) Why State Machine Replication is important for permissioned blockchain?

State Machine Replication (SMR) is **very important in permissioned blockchain** because it ensures that **all nodes maintain the same consistent state without using expensive mining.**

1. Ensures Consistency Across Nodes

- In permissioned blockchain, multiple known nodes maintain copies of the ledger
- SMR ensures that:
 - All nodes execute the **same transactions in the same order**
 - All nodes reach the **same final state**

→ This guarantees **data consistency and correctness**

2. Eliminates Need for Mining (Efficiency)

- Permissioned blockchains do **not use Proof-of-Work (PoW)**
- SMR allows consensus through **message passing instead of heavy computation**

→ Saves:

- Time
- Energy
- Computational resources

3. Works Well in Closed Network

- In permissioned systems:
 - Nodes are **known and trusted to some extent**
- This makes it feasible to:
 - Execute contracts on selected nodes
 - Use replication efficiently

→ SMR is ideal for **closed environments**

4. Fault Tolerance and Reliability

- Even if some nodes fail (crash faults or Byzantine faults), SMR ensures:
 - System continues to function
 - Correct state is preserved

→ Improves **reliability and availability**

5. Supports Deterministic Smart Contract Execution

- All nodes must produce the **same output for same input**
- SMR ensures:
 - Deterministic execution
 - Same results across all replicas

6. Faster Consensus

- Instead of mining, SMR uses:
 - Algorithms like **PBFT, Raft, Paxos**
- These are faster because:
 - Fewer nodes
 - Known participants

→ Leads to **high throughput and low latency**

Conclusion

State Machine Replication is crucial in permissioned blockchain because it provides **consistency, efficiency, fault tolerance, and fast consensus** without relying on energy-intensive mining. It is the backbone of how permissioned systems maintain a **synchronized and reliable distributed ledger**.



b

Q3 (b) Explain all steps for State Machine Replication

State Machine Replication (SMR) ensures that **all nodes execute the same operations in the same order**, so they reach the **same state**.

Steps in State Machine Replication

1. Replication of State Machines

- Multiple copies of the **same state machine** are placed on different nodes (servers)
- Each node maintains a **local replica of the system state**

2. Receiving Client Requests

- Clients send requests (transactions) to the system
- These requests act as **inputs to the state machine**

3. Propagation of Requests

- The received request is **broadcasted to all replicas (nodes)**
 - Ensures every node gets the same input
-

4. Ordering of Requests

- All nodes must process requests in the **same order**
- A **consensus/ordering algorithm** (like PBFT, Raft, Paxos) is used

→ This step is critical for consistency

5. Execution of Requests

- Each node executes the request **independently**
- Execution is done based on the **agreed order**

→ Since execution is deterministic, all nodes produce the same result

6. State Update

- Each node updates its **local state** after execution
 - All replicas reach the **same updated state**
-

7. Synchronization

- Nodes ensure their states remain **synchronized**
 - If any mismatch occurs, correction mechanisms are applied
-

Important Condition

- Execution must be **deterministic**
 - Same input + same order → same output
-

Conclusion

State Machine Replication works by **replicating state machines, agreeing on input order, and executing deterministically**, ensuring all nodes maintain a **consistent and synchronized state** in the system.



c

Q3 (c) If State Machine Replication is applied in public blockchain, what advantages and difficulties arise?

Applying **State Machine Replication (SMR)** in a **public (permissionless) blockchain** brings both benefits and challenges because public systems are **open, large-scale, and untrusted**.

Advantages

1. Faster Consensus

- SMR uses **message-based consensus (PBFT, Raft)** instead of mining
- Results in:
 - **Low latency**
 - Faster transaction confirmation

2. High Throughput

- No heavy PoW computation
- More transactions can be processed per second

3. Energy Efficiency

- Eliminates mining → **saves energy and cost**
- More environmentally friendly

4. Immediate Finality

- Once consensus is reached, transactions are **final**
- No need for multiple confirmations (unlike Bitcoin)

Difficulties / Challenges

1. Scalability Issues

- SMR requires **communication among nodes**
- In public blockchain (thousands of nodes), communication overhead becomes very high

2. Open Network (Unknown Participants)

- Public blockchain allows **anyone to join**
- SMR assumes **known and limited participants**
 - Hard to apply in fully open environments

3. Byzantine Fault Handling

- Public networks may contain **malicious nodes**
- SMR-based protocols (like PBFT) have limits:
 - Need **$3f + 1$ nodes to tolerate f faults**
 - Not efficient for large-scale open systems

4. Communication Overhead

- Consensus requires **multiple rounds of message exchange**
- In large networks → leads to **network congestion**

5. Leader-Based Bottleneck

- Many SMR protocols use a **leader node**
- If leader fails or is attacked → performance drops

6. Security Concerns

- Easier for attackers to:
 - Join network
 - Disrupt consensus

Conclusion

While State Machine Replication can improve **speed, efficiency, and finality**, it faces serious challenges in public blockchains due to **scalability, open participation, and high communication overhead**. Hence, SMR is more suitable for **permissioned blockchains**, while public blockchains prefer **PoW/PoS-based consensus**.



d

Q3 (d) Why distributed consensus? Explain distributed consensus with its requirements

Why Distributed Consensus is Needed

Distributed consensus is required because in a distributed system:

- There are **multiple nodes** maintaining copies of data
- Nodes may:
 - Fail (crash faults)
 - Behave maliciously (Byzantine faults)
 - Experience network delays

→ Yet, all nodes must **agree on a single correct value/state**

Main Reasons:

1. Agreement Among Nodes

- All nodes must **agree on the same data or decision**
 - Ensures consistency across the system
-

2. No Central Authority

- Blockchain systems are **decentralized**
 - No single controller → consensus is needed to coordinate
-

3. Fault Tolerance

- System should work even if some nodes fail
 - Consensus helps maintain correct operation
-

4. Data Consistency (Replication)

- Multiple copies of data exist
 - Consensus ensures all replicas have the **same state**
-

5. Real-world Examples

- Flight control systems
- Banking transactions

- Blockchain systems (Bitcoin, etc.)
-

Distributed Consensus (Explanation)

Distributed consensus is a process where **all correct nodes in a distributed system agree on a single value or decision**, even in the presence of faults.

Requirements of Distributed Consensus

1. Termination

- Every correct node must eventually **decide a value**
-

2. Agreement (Consistency)

- All correct nodes must agree on the **same value**
-

3. Validity

- The decided value must be **one of the proposed values**
-

4. Integrity

- A node decides **only once**
 - No fake or multiple decisions
-

Types of Faults Considered

- **Crash Faults** → node stops working
 - **Network Faults** → message delay/loss
 - **Byzantine Faults** → malicious or arbitrary behavior
-

Conclusion

Distributed consensus is essential to ensure **reliable, consistent, and fault-tolerant operation** in decentralized systems.

It guarantees that all nodes reach the **same decision**, even in the presence of failures, making it the foundation of blockchain and distributed computing.



4	a. Why do we require $3f+1$ replicas to ensure safety in an asynchronous system when there are f faulty nodes?	$2+3+1.5$ $+3.5=10$
	b. When view change is required? Explain in detail about View Change mechanism.	
	c. Explain a Use case, where permissioned bolckchian will be a suitable candidate with reason.	
	d. Explain Byzantine General Problem and its solution.	

a

Q4 (a) Why do we require $3f + 1$ replicas to ensure safety in an asynchronous system when there are f faulty nodes?

Basic Idea

To tolerate f **Byzantine (malicious) faults**, a distributed system must have at least:

$$3f + 1 \text{ replicas}$$

This ensures that **correct nodes can outvote faulty nodes** and still reach a safe consensus.

Reasoning

1. Presence of Faulty Nodes

- Out of total nodes, up to f can behave arbitrarily (maliciously)
- These nodes may:
 - Send incorrect messages
 - Send different messages to different nodes

2. Need for Majority of Honest Nodes

To ensure correctness:

- System must rely on **honest (non-faulty) nodes**
- Number of honest nodes:

$$(3f + 1) - f = 2f + 1$$

➡ So, at least $2f + 1$ **honest nodes exist**

3. Majority Voting Requirement

- For consensus, nodes typically require a **majority agreement**
- Majority condition:

$$\text{Majority} = 2f + 1$$

- Faulty nodes = f , which is **less than majority**

→ Honest nodes always **outnumber faulty nodes**

4. Avoiding Conflicting Decisions

- To prevent two groups from making **different decisions**, there must be overlap in voting sets
- Any two majority sets (size $2f + 1$) will have at least:

$$(2f + 1) + (2f + 1) - (3f + 1) = f + 1$$

common nodes

→ At least **one honest node is always common**, ensuring consistency

Why Not Less Than $3f + 1$?

- If fewer nodes are used:
 - Faulty nodes may dominate or split votes
 - Different groups may reach **different decisions**

→ **Consensus safety breaks**

Conclusion

We need $3f + 1$ **replicas** so that:

- There are always $2f + 1$ **honest nodes**
- Honest nodes form a **majority**
- Overlapping majorities ensure **no conflicting decisions**

Thus, $3f + 1$ guarantees **safety and consistency** in asynchronous systems with Byzantine faults.



b

Q4 (b) When is view change required? Explain the View Change mechanism

When is View Change Required?

A **view change** is required when the current **leader (primary node)** is **faulty or not functioning properly**.

Situations triggering view change:

- Leader **fails/crashes**
- Leader is **slow or unresponsive**
- Leader behaves **maliciously (Byzantine fault)**
- Requests are **not being processed within time (timeout occurs)**

→ In such cases, the system must **replace the faulty leader** to ensure progress.

What is View Change?

- The system is divided into **views**
 - Each view has a **designated primary (leader)**
 - When the primary fails → system moves to a **new view with a new leader**
-

View Change Mechanism (PBFT-based explanation)

1. Fault Detection (Timeout)

- Backup nodes wait for a response from the primary
 - If no response within a time limit → **suspect leader is faulty**
-

2. Trigger View Change

- Backup nodes send a **VIEW-CHANGE message** to other nodes
 - This message includes:
 - Current view number
 - State information (checkpoint, logs)
-

3. Collecting View-Change Messages

- When a node receives $2f + 1$ view-change messages:

- It confirms that majority nodes agree leader is faulty

4. Selecting New Leader

- A **new primary** is selected (based on view number, e.g., round-robin)
- New view number = old view + 1

5. New View Formation

- New leader sends a **NEW-VIEW message** to all replicas
- Includes:
 - Collected view-change messages
 - Updated state (to maintain consistency)

6. State Synchronization

- All nodes update their state to match the new view
- Ensures no transaction is lost or duplicated

7. Resume Execution

- System resumes normal operation with the **new leader**

Important Points

- Ensures **liveness** (system keeps making progress)
- Prevents system from getting stuck due to faulty leader
- Maintains **consistency and correctness**

Conclusion

View change is required when the leader fails or behaves incorrectly.

It allows the system to **replace the faulty leader, synchronize state, and continue execution**, ensuring both **liveness and fault tolerance** in distributed consensus systems.



c

Q4 (c) Explain a use case where permissioned blockchain is suitable, with reason

Use Case: Supply Chain Management

A **permissioned blockchain** is highly suitable for **supply chain tracking of goods** (e.g., food, pharmaceuticals, luxury items).

Scenario

Participants involved:

- Supplier
- Manufacturer
- Distributor
- Retailer

Each participant is a **known and authorized entity**.

They record:

- Production details
- Shipment data
- Ownership transfer
- Quality checks

All updates are stored on a **shared blockchain ledger**.

Why Permissioned Blockchain is Suitable

1. Known Participants

- All entities (supplier, distributor, etc.) are **identified and verified**
 Matches permissioned model where users are authenticated
-

2. Controlled Access

- Only authorized members can:
 - Read/write data
 Ensures **data privacy and security**
-

3. High Performance

- Faster consensus (PBFT/Raft)
- No mining overhead
- Suitable for real-time tracking

4. Data Integrity & Transparency

- Every transaction is recorded immutably
- Helps detect fraud and tampering

5. Traceability

- Product journey can be tracked:
 - From origin → final consumer
 - Useful in case of defects or recalls

6. Trust Among Participants

- Even if participants don't fully trust each other, blockchain ensures **shared truth**

Example

Tracking medicines:

- Prevents counterfeit drugs
- Ensures authenticity
- Maintains audit trail

Conclusion

Supply chain management is an ideal use case for permissioned blockchain because it involves **known participants, requires privacy, high efficiency, and trust**, all of which are well-supported by permissioned systems.



d

Q4 (d) Explain Byzantine General Problem and its solution

Byzantine General Problem

The **Byzantine General Problem** describes a situation in a distributed system where:

- Multiple nodes (generals) must **agree on a common decision** (e.g., attack or retreat)
- Some nodes may be **faulty or malicious (traitors)**
- Communication happens through messages

Problem Statement

- A commander sends an order to lieutenants
- Some nodes may:
 - Send **false messages**
 - Send **different messages to different nodes**
- Goal: All honest nodes must:
 - **Agree on the same decision**
 - Follow a **correct value if the commander is honest**

Challenges

- Faulty nodes can create **confusion and inconsistency**
- Honest nodes may receive **conflicting information**
- Simple majority may fail if faults are high

Conditions for Correct Solution

A correct solution must satisfy:

1. **Agreement:** All honest nodes agree on the same value
2. **Validity:** If the commander is honest, all follow his order

Solution to Byzantine Problem

1. Requirement: $3f + 1$ Nodes

- To tolerate f faulty nodes, system needs:

$$3f + 1 \text{ total nodes}$$

→ Ensures enough honest nodes for majority

2. Majority Voting Mechanism

- Nodes exchange messages with each other
 - After multiple rounds, decision is taken using **majority voting**
 - Faulty messages are outvoted by honest nodes
-

3. Message Exchange (Rounds)

- Commander sends message to all
 - Lieutenants share received values with each other
 - After sufficient rounds:
 - Nodes compare received values
 - Apply majority rule
-

4. Byzantine Fault Tolerant Algorithms

Modern systems use algorithms like:

- **PBFT (Practical Byzantine Fault Tolerance)**
 - Works in asynchronous systems
 - Uses phases: **Pre-prepare** → **Prepare** → **Commit**
 - Requires $3f + 1$ nodes
 - **Other models:**
 - Lamport's Byzantine algorithm
 - Tendermint, Hyperledger consensus
-

Key Insight

- Even if some nodes lie, the system still reaches **correct consensus**
 - Requires:
 - Sufficient number of honest nodes
 - Reliable communication
-

Conclusion

The Byzantine General Problem models **faulty and malicious behavior in distributed systems**. It is solved using **Byzantine Fault Tolerant algorithms with $3f + 1$ nodes and majority-based agreement**, ensuring **consistency and reliability** even in the presence of adversarial nodes.